

# CS-302 Artificial Intelligence

## Experiment # 5

Understanding and implementing K Mean Clustering algorithm

### Objective/s:

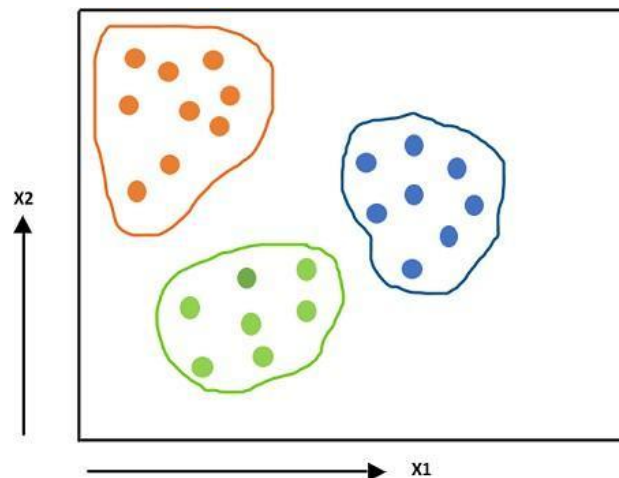
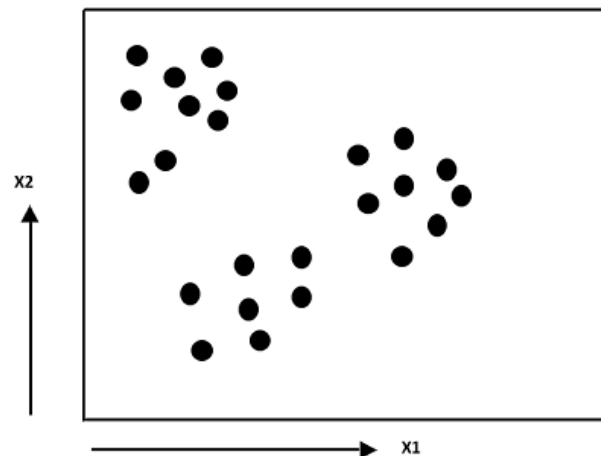
- Overview of Clustering
- Overview of K-Mean Clustering
- Python code implementation from scratch
- Lab Tasks

### 1. Overview of Clustering

In this article, we shall be covering the role of unsupervised learning algorithms, their applications, and K-means clustering approach. On a brief note, Machine learning algorithms can be classified as Supervised and unsupervised learning. In the supervised learning, there will be the data set with input features and the target variable. The aim of the algorithm is to learn the dataset, find the hidden patterns in it and predict the target variable. The target variable can be continuous as in the case of Regression or discrete as in the case of Classification. Examples of Regression problems include housing price prediction, Stock market prediction, Air humidity, and temperature prediction. Examples of classification problems include Cancer prediction(either benign or malignant), email spam classification etc. Our article demonstrates the next category of machine learning which is unsupervised learning.

So now one of the other areas of machine learning is unsupervised learning, where we will have the data, but we don't have any target variable as in the case of supervised learning. So the goal here is to observe the hidden patterns among the data and group them into clusters i.e the data points which have some shared properties will fall into one cluster or one alike group. So where is clustering used? Ever observed the google news where each category like sports, politics, movies, science have 1000's of news articles? This is clustering. The algorithm groups the news articles which have common features in them into separate groups to form a cluster. Other examples of clustering include social network analysis, market segmentation, recommendation systems etc.

One of the basic clustering algorithms is K-means clustering algorithm which we are going to discuss and implement from scratch in this article. Let's look at the final aim of the clustering from the two sample images and a practical example. The first image is the plot of the data set with features  $x_1$  and  $x_2$ . You can see that data is unclustered, so we can't conclude anything just by looking into the plot. The second image is obtained after performing Clustering, we can observe that data points which are close to each other are grouped as a cluster. In practical situations, this can be treated as a case where we are dealing with customer data and are asked to segment the market based on customer's income and the number of previous transactions. Suppose here  $x_1$  feature is the annual income and  $x_2$  feature is the number of transactions, based on these features we can cluster the data and segment them into three categories like customers with low annual income but a high number of transactions (like orange cluster), customers with medium income and a medium number of transactions (like green cluster), customers with high income but low number of transactions (like blue cluster). Based on these segments, the marketing team of the company can redefine their marketing strategies to get more transactions by recommending the products, which each cluster's customer might buy.



## 2. K-Mean Clustering

So far, we have discussed the goal of clustering and a practical application, now it's time to dive into K-means clustering implementation and algorithm. As the name itself suggests, this algorithm will regroup  $n$  data points into  $K$  number of clusters. So given a large amount of data, we need to cluster this data into  $K$  clusters.

But the problem is how to choose the number of clusters? Most of the times we will know what type of clusters we need to use, for example in the case of Google news, we already knew that say 4 types of clusters (sports, politics, science, movies) exist and news articles should be clustered into any of them. But in the case of market segmentation problem, as the goal is to cluster the type of customers, we don't know how many types of customers are present in the dataset (like rich, poor, who loves shopping, who doesn't love shopping etc) i.e we don't know exactly how many number of clusters that need to be chosen and we can't randomly assume the number of clusters based on some ground rules (like people with annual income greater than ' $x$ ' amount should be one cluster and lower than ' $x$ ' should fall into another).

We shall discuss the solution to this problem, later in the article but now let's assume that we are going to segment our data into 3 clusters. Some of the mathematical terms involved in K-means clustering are centroids, Euclidean distance. On a quick note centroid of a data is the average or mean of the data and

Euclidian distance is the distance between two points in the coordinate plane. Given two points A(x<sub>1</sub>,y<sub>1</sub>) and B(x<sub>2</sub>,y<sub>2</sub>), the Euclidian distance between these two points is :

$$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

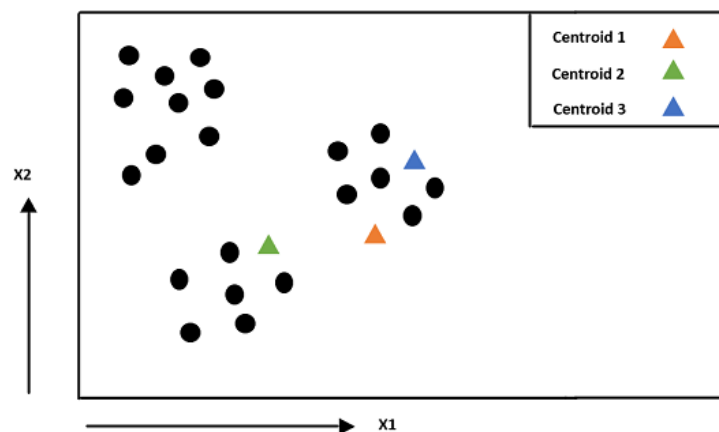
While implementing, we won't consider the square root as we can compare the square of the distances and arrive at conclusions.

## 2.1 Algorithm

Now let's start talking about the implementation steps. We shall be using either cluster centers or centroids words to describe the cluster centers.

### Step 1:

Randomly initialize the cluster centers of each cluster from the data points. In fact, random initialization is not an efficient way to start with, as sometimes it leads to increased numbers of required clustering iterations to reach convergence, a greater overall runtime, and a less-efficient algorithm overall. So there are many techniques to solve this problem like K-means++ etc. Let's also discuss one of the approaches to solve the problem of random initialization later in the article. So let's assume K=3, so we choose randomly 3 data points and assume them as centroids.

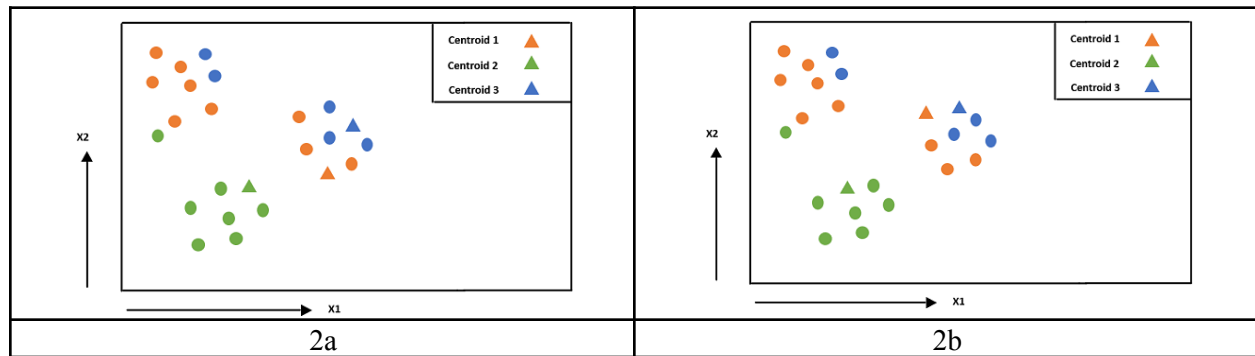


Here three cluster centers or centroids with the green, orange, and blue triangle markers are chosen randomly. Again, this is not an efficient method to choose the initial cluster centers.

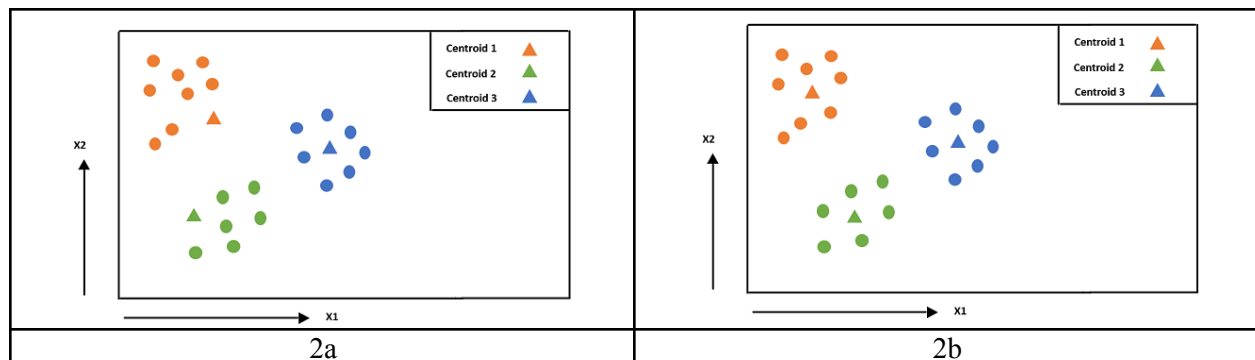
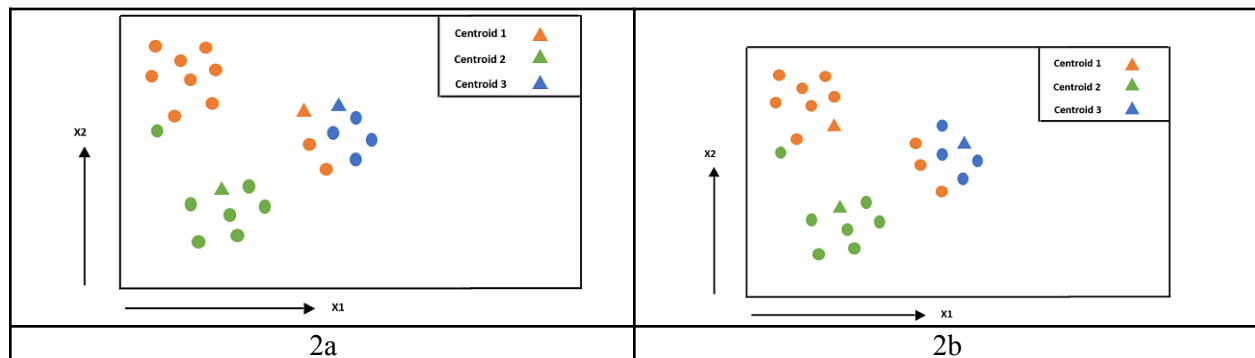
### Step 2:

**2a.** For each data point, compute the Euclidian distance from all the centroids (3 in this case) and assign the cluster based on the minimal distance to all the centroids. In our example, we need to take each black dot, compute its Euclidian distance from all the centroids (green, orange, and blue), and finally color the black dot to the color of the closest centroid.

**2b.** Adjust the centroid of each cluster by taking the average of all the data points which belong to that cluster based on the computations performed in step 2a. In our example, as we have assigned all the data points to one of the clusters, we need to calculate the mean of all the individual clusters and move the centroid to calculated mean. Repeat this process till clusters are well separated or convergence is achieved.



Repeat these steps until convergence is achieved:



## 2.2 Implementation from scratch:

Now as we are familiar with intuition, let's implement the algorithm in python from scratch. We need NumPy, pandas and matplotlib libraries to improve the computational complexity and visualization of the results. The dataset which we are going to use is 'Mall\_Customers.csv' and the link to the dataset can be found in the GitHub page. (Note: The dataset has been downloaded from SuperDataScience website).

Let's read the dataset and get the data examples

```
dataset=pd.read_csv('Mall_Customers.csv')  
dataset.describe()
```

|       | CustomerID | Age        | Annual Income (k\$) | Spending Score (1-100) |
|-------|------------|------------|---------------------|------------------------|
| count | 200.000000 | 200.000000 | 200.000000          | 200.000000             |
| mean  | 100.500000 | 38.850000  | 60.560000           | 50.200000              |
| std   | 57.879185  | 13.969007  | 26.264721           | 25.823522              |
| min   | 1.000000   | 18.000000  | 15.000000           | 1.000000               |
| 25%   | 50.750000  | 28.750000  | 41.500000           | 34.750000              |
| 50%   | 100.500000 | 36.000000  | 61.500000           | 50.000000              |
| 75%   | 150.250000 | 49.000000  | 78.000000           | 73.000000              |
| max   | 200.000000 | 70.000000  | 137.000000          | 99.000000              |

For visualization convenience, we are going to take Annual Income and Spending score as our data.

```
X = dataset.iloc[:, [3, 4]].values
```

Now X is two matrix of shape (200,2).

Next step is to choose the number of iterations which might guarantee convergence. We need to try many possibilities to find optimum number of iterations required for convergence. There is no need to choose a very large number because say at 100th iteration, if the centroids arrived at their true location or best possible location, even after performing 1000 extra iterations, the algorithm will give same results. So, for convenience let's start by choosing number of iterations as 100.

```
m=X.shape[0] #number of training  
examples n=X.shape[1] #number of  
features. Here n=2n_iter=100
```

Next step is to choose number of clusters K. Let's take 5 as K and as it has been mentioned earlier, we are going to see a method later in the article, which will find us the optimum number of clusters K.

```
K=5 # number of clusters
```

We are ready to implement our Kmeans Clustering steps. Let's proceed:

**Step 1:** Initialize the centroids randomly from the data points:

```
Centroids=np.array([]).reshape(n,0)
```

Centroids is a n x K dimensional matrix, where each column will be a centroid for one cluster.

```
for i in range(K):  
    rand=rd.randint(0,m-1)  
    Centroids=np.c_[Centroids  
    ,X[rand]]
```

**Step 2.a:** For each training example compute the Euclidian distance from the centroid and assign the cluster based on the minimal distance. The output of our algorithm should be a dictionary with cluster number as Keys and the data points which belong to that cluster as values. So, let's initialize the dictionary.

```
Output={}
```

We find the Euclidian distance from each point to all the centroids and store in a  $m \times K$  matrix. So, every row in EuclidianDistance matrix will have distances of that particular data point from all the centroids. Next, we shall find the minimum distance and store the index of the column in a vector C.

```
EuclidianDistance=np.array([]).res  
hape(m,0) for k in range(K):  
    tempDist=np.sum((X-Centroids[:,k])**2,axis=1)  
    EuclidianDistance=np.c_[EuclidianDistance,tempDi  
st]  
C=np.argmin(EuclidianDistance,axis=1)+1
```

**Step 2.b:** We need to regroup the data points based on the cluster index C and store in the Output dictionary and compute the mean of separated clusters and assign it as new centroids. Y is a temporary dictionary which stores the solution for one iteration.

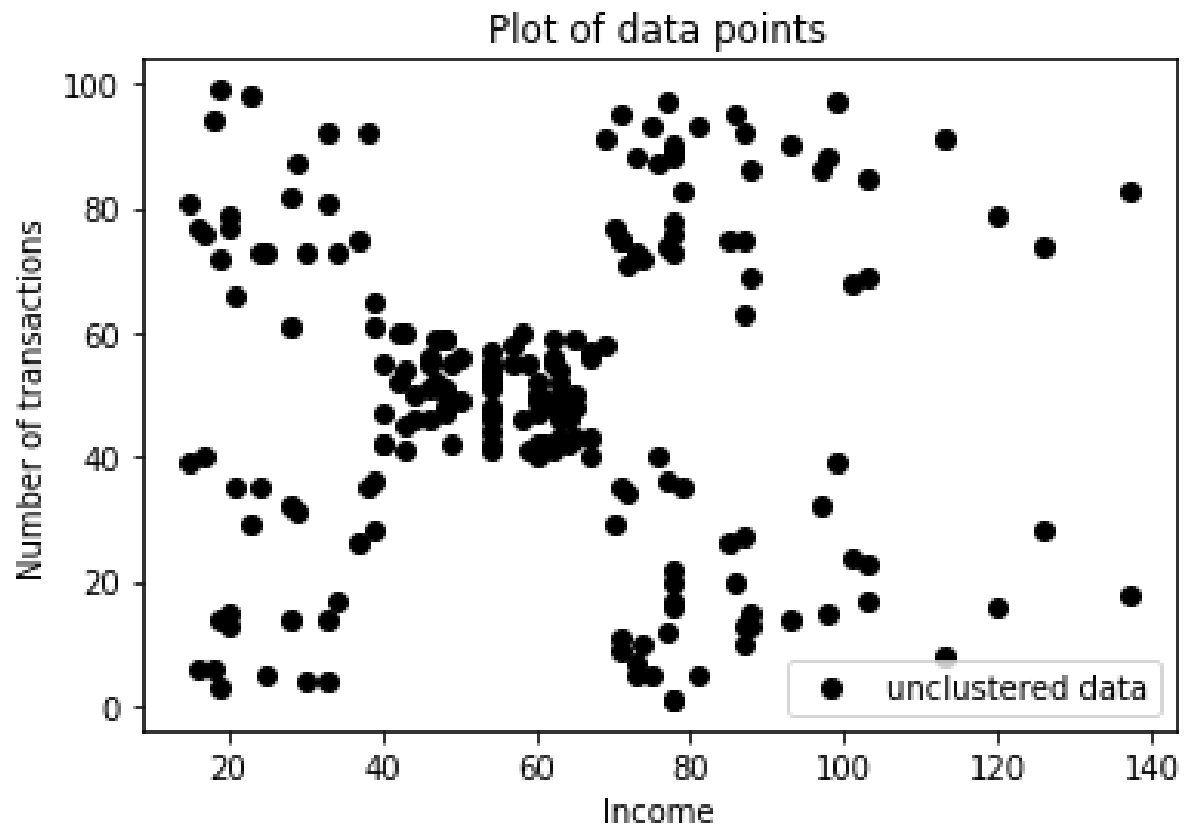
```
Y={}  
for k in range(K):  
    Y[k+1]=np.array([]).resha  
pe(2,0)  
for i in range(m):  
    Y[C[i]]=np.c_[Y[C[i]],X[i]  
1]  
  
for k in range(K):  
Y[k+1]=Y[k+1].T  
  
for k in range(K):  
    Centroids[:,k]=np.mean(Y[k+1],axis=0  
)
```

Now we need to repeat step 2 till convergence is achieved. In other words, we loop over  $n\_iter$  and repeat the steps 2.a and 2.b as shown:

```
for i in  
    range(n_iter)  
    : #step 2.a  
EuclidianDistance=np.array([]).reshap  
e(m,0) for k in range(K):  
tempDist=np.sum((X-Centroids[:,k])**2,axis=1)  
    EuclidianDistance=np.c_[EuclidianDistance,tempD  
ist]  
    C=np.argmin(EuclidianDistance,ax  
is=1)+1 #step 2.b  
Y={}  
    for k in range(K):  
        Y[k+1]=np.array([]).reshape(2,0)  
    for i in range(m):  
        Y[C[i]]=np.c_[Y[C[i]  
],X[i]]  
  
for k in range(K):  
Y[k+1]=Y[k+1].T  
  
    for k in range(K):  
        Centroids[:,k]=np.mean(Y[k+1],axi  
s=0)  
Output=Y
```

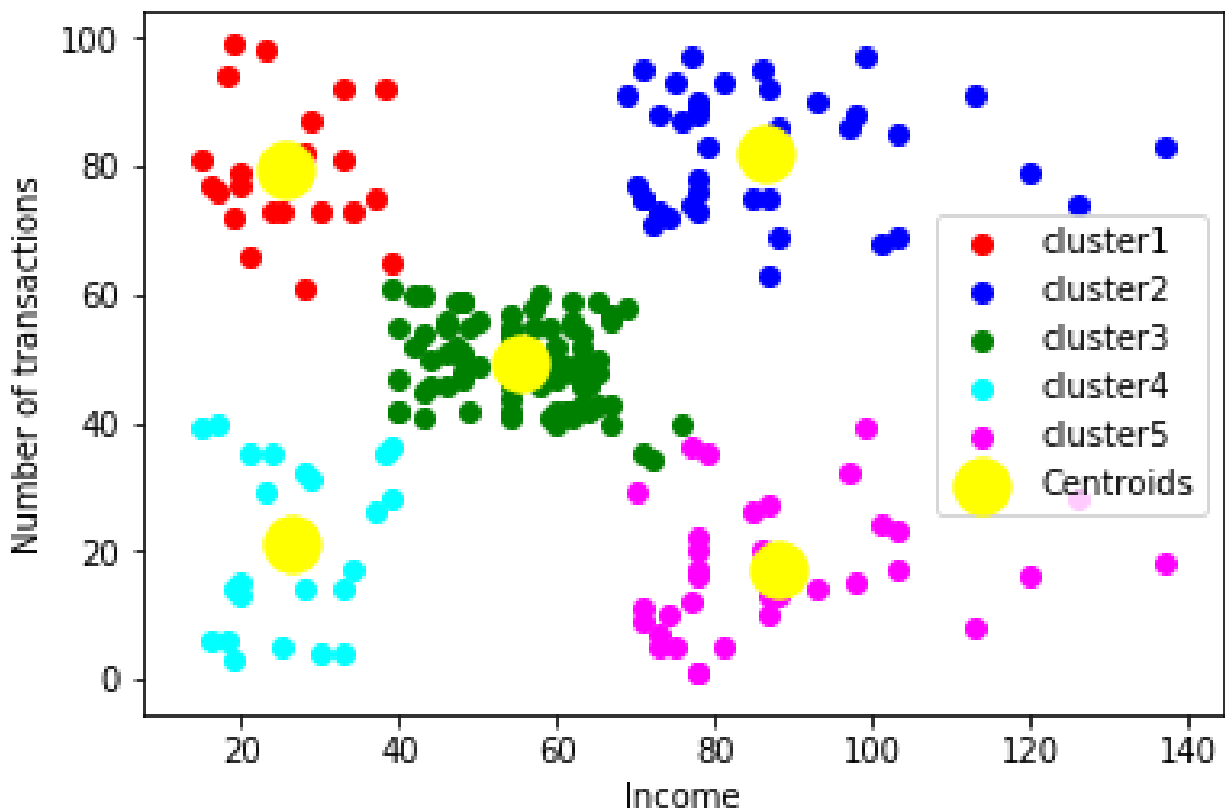
Now it's time to visualize the algorithm and notice how the original data is clustered. To start with, let's scatter the original unclustered data first.

```
plt.scatter(X[:,0],X[:,1],c='black',label='unclustered  
data') plt.xlabel('Income')  
plt.ylabel('Number of  
transactions') plt.legend()  
plt.title('Plot of data  
points') plt.show()
```



Now let's plot the clustered data:

```
color=['red','blue','green','cyan','magenta']  
labels=['cluster1','cluster2','cluster3','cluste  
r4','cluster5'] for k in range(K):  
  
plt.scatter(Output[k+1][:,0],Output[k+1][:,1],c=color[  
k],label=labels[k])  
plt.scatter(Centroids[0,:],Centroids[1,:],s=300,c='yel  
low',label='Centroi ds')  
plt.xlabel('Income')  
plt.ylabel('Number of  
transactions') plt.legend()  
plt.show()
```

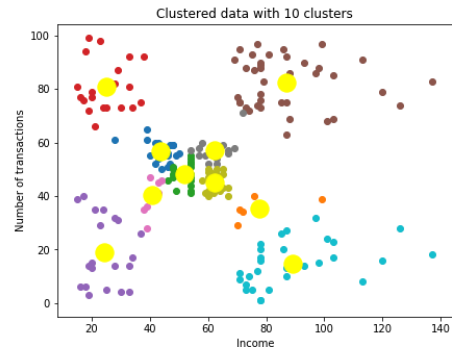
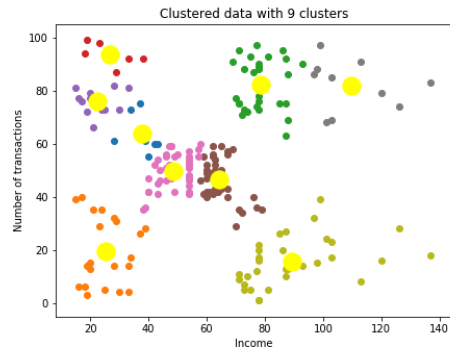
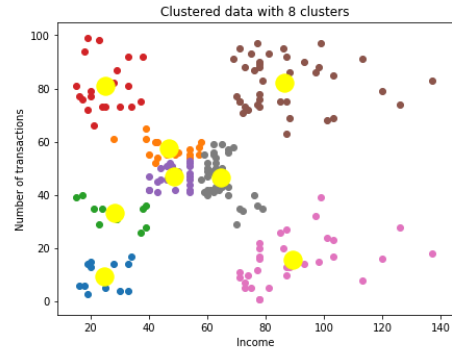
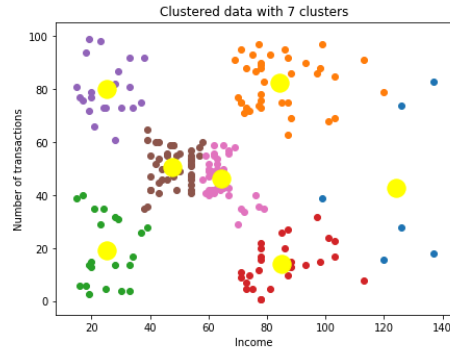
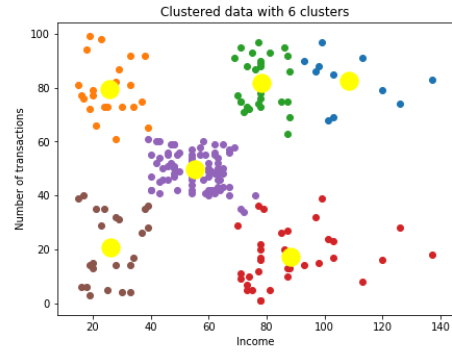
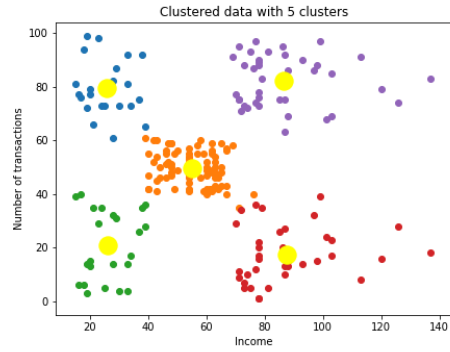
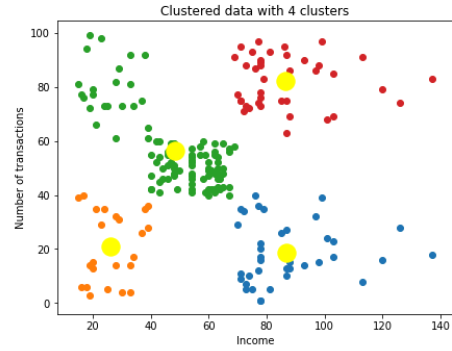
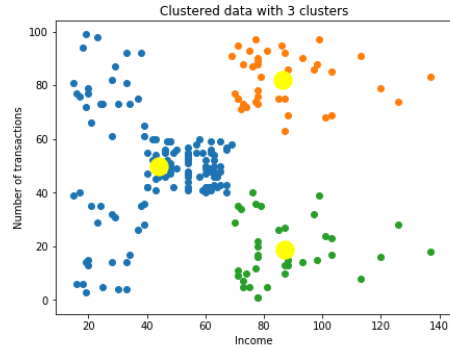
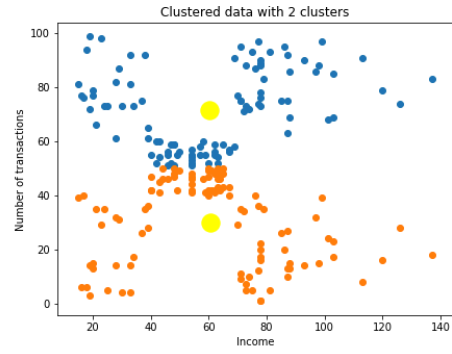


Wow, it's so beautiful and informative too! Our data has become a clustered data from unclustered raw data. We can observe that there are five categories of clusters which are :

1. Customers with low income but a high number of transactions (For this type may be the company can recommend products with low price) — Red cluster
2. Customers with low income and a low number of transactions (Maybe these types of customers are too busy saving their money) — Cyan Cluster
3. Customers with medium income and a medium number of transactions — Green Cluster
4. Customers with High income and a low number of transactions — Magenta Cluster
5. Customers with High income and a high number of transactions — Blue cluster.

So, the company can divide their customers into 5 classes and design different strategies for a different type of customers to increase their sales. So far so good right, but how do we arrive at the conclusion that there should be 5 number of clusters? To find out how, let's visualize the clustered data with different clusters starting from 1 to 10.





Now we see a lot of plots showing the clustered data with a different number of clusters. So, the question is what is the best value for K? Suppose we have  $n$  data points, and we choose  $n$  clusters i.e., every data point is a cluster, is it a good model? No, obviously not and the converse is also not a good model i.e., one cluster for  $n$  data points. So how do we find the appropriate K? The answer lies in the fact that for every data point, its cluster center should be the nearest or in other words, “**Sum of squares of distances of every data point from its corresponding cluster centroid should be as minimum as possible**”. This statement will again contradict with the fact that if all data points are treated as individual clusters, then the sum of squares of distances will be 0. So, to counter this fact, we use a method called ELBOW method to find the appropriate number of clusters. The parameter which will be taken into consideration is **Sum of squares of distances of every data point from its corresponding cluster centroid** which is called **WCSS ( Within-Cluster Sums of Squares)**.

Steps involved in ELBOW method are:

1. Perform K means clustering on different values of K ranging from 1 to any upper limit. Here we are taking the upper limit as 10.
2. For each K, calculate WCSS
3. Plot the value for WCSS with the number of clusters K.
4. The location of a bend (knee) in the plot is generally considered as an indicator of the appropriate number of clusters. i.e., the point after which WCSS doesn't decrease more rapidly is the appropriate value of K.

### 3. Python code implementation from scratch

### 4. Tasks

1. Write the above-mentioned program for different datasets.